MINI-PROJECT: SkillSwap

Teach what you know, learn what you need • time credits, not money

1. Overview

Problem: Many students are good at one thing (Python, LaTeX, Figma, Excel, spoken English) and stuck on another. Finding a peer to learn from is hard, and paid tutoring is expensive.

Goal: An app where students list skills they can teach and want to learn, tutors publish session slots, learners book with time credits (1 hour = 1 credit), both confirm after the session, and credits move safely.

In scope (MVP)	Out of scope (for now)
Profile with offers and wants • tutor slots • search by skill • book (credit check) • cancel ≥ 2 h before • both-sides confirmation → credit transfer • ratings • credit ledger • suggested matches	Money • online video calls in the app • exam or assignment help

2. Team roles

Role	Owns	In this project
R1: Team Lead & Product Owner	Plan, requirements, user interviews, daily stand-up, pitch	Recruits first 20 tutors; owns credit and integrity rules; pitch
R2: UI/UX Designer	Figma screens, Designer view, colours, accessibility, app icon	Profile and slot cards, credit wallet, rating screen
R3: App Developer (Blocks)	Master App Inventor project, all blocks logic, APK builds	Search, booking, confirmations, APK
R4: Backend & Data Engineer	Google Sheet, Apps Script API, data quality, dashboard data	Users, Slots, Ledger, Ratings tabs; safe credit transfer with LockService
R5: QA, Integration & Outreach	Test cases, bug list, Google Site, user testing, social post, demo video	Test plan (credit edge cases), Google Site, pilot for 2 weeks

How 5 people work on one App Inventor project

- **One master project** owned by R3. Others build in their own copies.
- To move blocks between accounts: right-click a block → **Download Blocks as PNG**, send the PNG, and **drag the PNG into the other workspace**: the blocks appear.
- Every evening R3 exports the master `.aia` to the shared Drive folder as `ProjectName_DayN.aia` (daily backup).
- **10-minute stand-up** each day: what I did, what I'll do, what blocks me. R1 writes it in the progress log (Day | Done | Next | Blockers).

3. Requirements

3A. User stories

#	As a...	I want to...	So that...
US1	student	list skills I can teach and want to learn	others can find me
US2	tutor	publish slots with a place or online link	learners can book
US3	learner	search for 'LaTeX' and book a slot	I learn from a peer
US4	learner	see my credit balance	I know what I can book
US5	both	confirm the session happened	credits move fairly
US6	student	rate my session	good tutors get noticed

3B. Functional requirements

ID	Requirement	Priority	Acceptance test
FR-1	Sign up with university email; start with **2 credits**	Must	New user has 2 credits
FR-2	Edit offers/wants (comma-separated skills)	Must	Users tab updated

ID	Requirement	Priority	Acceptance test
FR-3	Tutor creates a slot: skill, date, time, place/link	Must	Slots row open
FR-4	Search open future slots by skill (not my own)	Must	Only matching slots
FR-5	Book: need ≥ 1 available credit (credits − pending bookings)	Must	0 credits → *not enough credits*
FR-6	Cancel up to 2 h before start	Should	After that → *too late to cancel*
FR-7	Both tutor and learner confirm → 1 credit learner → tutor (lock + ledger)	Must	Balances change once only
FR-8	Rate after confirmation	Should	Ratings row

3C. Non-functional requirements

- Credit changes only on the **server**, under a **lock**, and every change has a **Ledger** row.
- Sessions in **public campus places** or online only.
- Profiles show nickname and skills, not phone numbers.
- Integrity rule shown on booking: learning only, no doing assignments or exams for others.

4. Design

4A. Architecture

App → **Apps Script API** → Sheet (Users, Slots, Ledger, Ratings). Credits computed by the server; **LockService** prevents double transfers. **Looker Studio**: most-wanted skills and sessions per week.

4B. Screens (one VerticalArrangement per page)

Page	Purpose	Main components (name: type)
login	Sign in	txtEmail, txtNick • btnLogin
profile	My profile	txtOffers, txtWants • lblCredits • btnSave
search	Find a tutor	txtSkill • btnSearch • lvSlots
slot	Slot details	lblSkill, lblTutor, lblWhen, lblPlace • btnBook
teach	Publish a slot	txtSlotSkill • dpDate • tpTime • txtPlace • btnPublish
sessions	My sessions	lvSessions • btnConfirm, btnCancel, btnRate
(non-visible)		webApi • TinyDB1 • Notifier1

4C. Data model (Google Sheet tabs)

Tab	Columns (row 1)	Notes
Users	email | nick | offers | wants | credits	
Slots	slotId | tutor | skill | date | time | place | status | learner | tutorOk | learnerOk	status = open / booked / done / cancelled
Ledger	time | from | to | credits | slotId	
Ratings	slotId | from | stars | time	

4D. API (Apps Script web app)

action	Extra parameters	Returns
login	&email &nick	{credits}
profile	&email &offers &wants	{ok}
search	&email &skill	open future slots
publish	&email &skill &date &time &place	{slotId}
book	&email &slot	{ok} / {error}
cancel	&email &slot	{ok}
confirm	&email &slot	{ok, transferred}
mine	&email	my slots as tutor or learner + credits
profile / rate	&email &offers &wants / &slot &stars	{ok}

5. Building the working application

5A. Sheet and backend (R4)

1. Create the Google Sheet with the tabs in 4C (exact names, row 1 headings).
2. **Extensions → Apps Script**. Paste the common helpers, then the project code below.
3. **Deploy → New deployment → Web app**, *Execute as* Me, *Who has access* Anyone → Authorize → copy the `/exec` URL.
4. Test each action in Chrome, e.g. **…**`/exec?key=team-secret-123&action=login`.
5. After **every** code change: Deploy → Manage deployments → Edit → **New version**.

```
// ---------- common helpers (paste once at the top of Code.gs) ----------
const KEY = 'team-secret-123';          // change this; same value in the app

function doGet(e) {
  const p = e.parameter;
  if (p.key !== KEY) return out(p.action, {error: 'bad key'});
  const ss = SpreadsheetApp.getActiveSpreadsheet();
  try { return out(p.action, route(ss, p)); }
  catch (err) { return out(p.action, {error: String(err)}); }
}
function rows(ss, tab) {               // a tab as a list of objects
  const v = ss.getSheetByName(tab).getDataRange().getDisplayValues();
  const h = v.shift();
  return v.map(r => Object.fromEntries(h.map((k, i) => [k, r[i]])));
}
function out(action, data) {           // every reply says which action it answers
  return ContentService.createTextOutput(JSON.stringify({action: action, data: data}))
         .setMimeType(ContentService.MimeType.JSON);
}
```

```
// ---------- SkillSwap routes (key parts) ----------
function route(ss, p) {
  const email = (p.email || '').toLowerCase();
  let me = rows(ss, 'Users').find(u => u.email === email);
  if (p.action === 'login') {
    if (!email.endsWith('@uohyd.ac.in')) return {error: 'Use your university email'};
    if (!me) { ss.getSheetByName('Users').appendRow([email, p.nick, '', '', 2]); return
{credits: 2}; }
    return {credits: me.credits};
  }
  if (!me) return {error: 'Please log in again'};
  const slots = rows(ss, 'Slots');
  if (p.action === 'search') return slots.filter(s => s.status === 'open' && s.tutor !==
email &&
      s.skill.toLowerCase().includes(String(p.skill).toLowerCase()) && new Date(s.date +
'T' + s.time) > new Date());
  if (p.action === 'profile') { const sh = ss.getSheetByName('Users'), v =
sh.getDataRange().getValues();
    for (let i = 1; i < v.length; i++) if (v[i][0] === email) sh.getRange(i + 1, 3, 1,
2).setValues([[p.offers, p.wants]]);
    return {ok: true}; }
  if (p.action === 'rate') { ss.getSheetByName('Ratings').appendRow([p.slot, email,
Number(p.stars), new Date()]); return {ok: true}; }
  if (p.action === 'publish') { const id = 'SL' + Date.now();
    ss.getSheetByName('Slots').appendRow([id, email, p.skill, p.date, p.time, p.place,
'open', '', '', '']); return {slotId: id}; }
  if (p.action === 'book') {
    const pending = slots.filter(s => s.learner === email && s.status ===
'booked').length;
    if (Number(me.credits) - pending < 1) return {error: 'Not enough credits: teach a
session to earn more'};
    return setSlot(ss, p.slot, s => s.status === 'open' && s.tutor !== email, {status:
'booked', learner: email});
  }
  if (p.action === 'cancel') return setSlot(ss, p.slot, s => s.status === 'booked' &&
(s.learner === email || s.tutor === email) &&
      new Date(s.date + 'T' + s.time) - new Date() > 2 * 36e5, {status: 'open', learner:
''});
  if (p.action === 'confirm') return confirm(ss, p.slot, email);
  if (p.action === 'mine') return {credits: me.credits, slots: slots.filter(s => s.tutor
=== email || s.learner === email)};
  return {error: 'unknown action'};
}
function confirm(ss, slotId, email) {
  const lock = LockService.getScriptLock(); lock.waitLock(10000);
  try {
    const sh = ss.getSheetByName('Slots'), v = sh.getDataRange().getDisplayValues(), h =
v[0];
    for (let i = 1; i < v.length; i++) if (v[i][0] === slotId) {
      const s = Object.fromEntries(h.map((k, j) => [k, v[i][j]]));
      if (s.status !== 'booked') return {error: 'Session is not in booked state'};
      const col = email === s.tutor ? 'tutorOk' : email === s.learner ? 'learnerOk' :
null;
      if (!col) return {error: 'Not your session'};
      sh.getRange(i + 1, h.indexOf(col) + 1).setValue('yes'); s[col] = 'yes';
      if (s.tutorOk === 'yes' && s.learnerOk === 'yes') {
        sh.getRange(i + 1, h.indexOf('status') + 1).setValue('done');
        addCredits(ss, s.learner, -1); addCredits(ss, s.tutor, 1);
        ss.getSheetByName('Ledger').appendRow([new Date(), s.learner, s.tutor, 1,
slotId]);
        return {ok: true, transferred: true};
      }
      return {ok: true, transferred: false};
    }
    return {error: 'Unknown slot'};
  } finally { lock.releaseLock(); }
}
function addCredits(ss, email, n) {
  const sh = ss.getSheetByName('Users'), v = sh.getDataRange().getValues();
  for (let i = 1; i < v.length; i++) if (v[i][0] === email) sh.getRange(i + 1,
5).setValue(Number(v[i][4]) + n);
}
```

```
function setSlot(ss, id, allowed, fields) {
  const sh = ss.getSheetByName('Slots'), v = sh.getDataRange().getDisplayValues(), h =
v[0];
  for (let i = 1; i < v.length; i++) if (v[i][0] === id) {
    const s = Object.fromEntries(h.map((k, j) => [k, v[i][j]]));
    if (!allowed(s)) return {error: 'Not possible now (taken, your own slot, or too
late)'};
    Object.keys(fields).forEach(k => sh.getRange(i + 1, h.indexOf(k) +
1).setValue(fields[k]));
    return {ok: true};
  }
  return {error: 'Unknown slot'};
}
```

Security note: the key stops casual misuse only; anyone who unpacks the APK can find it. Never store passwords, phone numbers of others or ID numbers in this Sheet.

5B. Designer (R2)

- Pages: `vaLogin`, `vaProfile`, `vaSearch`, `vaSlot`, `vaTeach`, `vaSessions`; bottom bar Search / Teach / Sessions / Profile.
- Format Slots *date* (`yyyy-MM-dd`) and *time* (`HH:mm`) columns as Plain text.
- Wallet label on every page: ⏱️ `3 credits`.
- Booking page shows the integrity rule: *For learning only: no doing assignments or exams for others.*

5C. Blocks (R3)

Step 1: the API pattern (same in every project):

```
initialize global apiUrl to "https://script.google.com/macros/s/…/exec"
initialize global apiKey to "team-secret-123"

to callApi  action  params          ← params like "&user=Ravi&code=1" (already encoded)
  set webApi.Url to join get global apiUrl "?key=" get global apiKey "&action=" get action
get params
  call webApi.Get

when webApi.GotText (url, responseCode, responseType, responseContent)
  if responseCode ≠ 200  then  call Notifier1.ShowAlert "Network problem, try again"
  else initialize local r to call webApi.JsonTextDecodeWithDictionaries get
responseContent
       initialize local a to get value for key "action" in dictionary get r
       initialize local d to get value for key "data"   in dictionary get r
       in: call handleReply  get a  get d       ← one procedure decides what to do per
action
```

Step 2: pages

```
to showPage p
  set vaLogin.Visible to  compare texts  get p = "login"
  set vaProfile.Visible to  compare texts  get p = "profile"
  set vaSearch.Visible to  compare texts  get p = "search"
  set vaSlot.Visible to  compare texts  get p = "slot"
  set vaTeach.Visible to  compare texts  get p = "teach"
  set vaSessions.Visible to  compare texts  get p = "sessions"
```

Step 3: profile and search

```
when btnSave.Click  call callApi "profile" … "&offers=" UriEncode txtOffers.Text "&wants="
UriEncode txtWants.Text
when btnSearch.Click call callApi "search" … "&skill=" UriEncode txtSkill.Text
to handleReply a d
  if dictionary d has key "error" then ShowAlert (value "error" in d) • stop
  if a = "search" then set global slots to d • lines: skill " • " date " " time " • "
place → lvSlots
when lvSlots.AfterPicking  set global slot to … • fill labels • showPage "slot"
```

Step 4: publish and book

```
when btnPublish.Click  call callApi "publish" … "&skill=" … "&date=" dpDate (yyyy-MM-dd)
"&time=" tpTime (HH:mm) "&place=" …
when btnBook.Click
  call Notifier1.ShowChooseDialog "Book this session for 1 credit? (Learning only, no
assignment help)" "Confirm" "Book" "Cancel" false
when Notifier1.AfterChoosing (choice)  if choice = "Book" then call callApi "book" …
"&slot=" slotId
```

✔️ **YOU SHOULD SEE:** With 0 available credits → *Not enough credits*.

Step 5: my sessions, confirm, rate

```
when btnSessionsTab.Click  call callApi "mine" …
(in handleReply) if a = "mine" then wallet ← credits • lvSessions ← skill, date, role
(Teaching/Learning), status
when btnConfirm.Click  call callApi "confirm" … "&slot=" selected
(in handleReply) if a = "confirm" then
   if value "transferred" in d then ShowAlert "Both confirmed: 1 credit moved ✔" else
ShowAlert "Waiting for the other person to confirm"
when btnCancel.Click call callApi "cancel" … • btnRate → stars → call callApi "rate" …
```

6. Testing (R5)

#	Test	Expected	Pass?
T1	New user	2 credits	
T2	Book with 2 credits and 2 pending bookings	*Not enough credits*	
T3	Book your own slot	*Not possible now*	
T4	Cancel 3 h before	Slot open again	
T5	Cancel 1 h before	*too late*	
T6	Only learner confirms	No transfer yet	
T7	Both confirm	Learner −1, tutor +1, one Ledger row	
T8	Confirm again	*not in booked state* (no double transfer)	

7. Day-by-day plan (15 working days)

Day	Goal and checkpoint	Tasks by role
1	Kick-off. Checkpoint: roles signed, shared Drive folder, master project created	R1: read brief, set up WhatsApp group and Drive folder • R2: collect 3 reference apps • R3: create master project `SkillSwap_Team` • R4: create the Google Sheet with tabs • R5: create test-case and bug-list sheets
2	Understand users. Checkpoint: 5+ interviews done	All: interview students who could teach (LaTeX, Python, Figma, Excel, English) and learners (R1 prepares 6 questions) • R1: write the problem statement • R5: note quotes and pain points
3	Requirements. Checkpoint: user stories and MVP list agreed	R1: finalise user stories and acceptance criteria (Section 3) • R2: paper sketches • R4: data model (Section 4C) • R3: list components needed • R5: write test cases from acceptance criteria
4	Design. Checkpoint: Figma screens and seeded Sheet	R2: Figma screens for all pages • R4: fill Sheet with real seed data (20 tutors' skills, 15 slots over 2 weeks, test users) • R3: build empty pages and `showPage` • R1/R5: review designs with 2 users
5	Backend live. Checkpoint: API URL returns JSON in Chrome	R4: paste Apps Script, deploy, test every action in Chrome • R3: `callApi` and `handleReply` blocks • R2: finish Designer properties • R5: test API URLs from the test sheet
6	First data in the app. Checkpoint: app shows data from the Sheet	R3: login with starting credits; profile • R4: fix data issues • R2: polish list layout • R5: test on 2 phones • R1: update progress log
7	Core feature 1. Checkpoint: demo to TA	R3 + R2: publish slots; search by skill • R4: server side for it • R5: test cases • R1: stand-up and blockers
8	Core feature 2	R3 + R2: booking with credit check and cancel rule • R4: server side • R5: test • R1: start Google Site text
9	Core feature 3 + saving to backend	R3: two-sided confirmation, credit transfer, ratings • R4: validation rules on the server • R5: negative tests (empty input, no internet)

Day	Goal and checkpoint	Tasks by role
10	Integration. Checkpoint: **APK v0.1** installed on 3 phones	R3: build APK • R5: full test run, bug list • R4: dashboard data tab • R1 + R2: Google Site with QR to the APK
11	Pilot. Checkpoint: **10 real users** tried it	R1 + R5: run pilot (two weeks: at least 10 sessions confirmed by both sides), feedback form • R3/R4: on-call fixes • R2: note UI confusion
12	Fix and improve	All: fix the **top 3** problems from feedback • R3: advanced feature 1: suggested matches (my wants ↔ others' offers)
13	Advanced + dashboard	R3: advanced feature 2: most-wanted skills dashboard • R4: Looker Studio dashboard • R5: retest everything
14	Policy and polish. Checkpoint: final APK	R1: one-page privacy and safety policy • R2: icon, screenshots • R3: final APK + .aia • R5: 2-minute demo video • R4: API backup copy
15	Pitch and submit	R1: Beamer pitch (with R2) • All: rehearse twice • R5: submission checklist • Demo Day / submission

8. Safety, privacy and ethics

- Meet only in **public campus places** (library, department labs) or online; never in rooms.
- **No money** and no paid tutoring through the app.
- **Academic integrity:** sessions are for learning; never for doing someone's assignments, lab records or exams. Violations lead to removal.
- Report button for misconduct; admin can block users.

Deliverables and marks (100)

`.aia` (final) • APK • Apps Script code + API URL • Google Sheet (view link) • Google Site • dashboard • progress log • privacy and safety policy • Beamer pitch • 2-min demo video • pilot feedback.

Problem evidence	MVP working	Advanced (2+)	Real users (10+)	Privacy policy	Dashboard	Pitch + video
15	25	15	15	10	10	10

Individual marks: 70% team score + 30% individual (TA checks each role's work in the progress log and a 3-minute viva per member).